What Are Evolutionary Algorithms?

Evolutionary Algorithms (EAs) are population-based search and optimization methods inspired by biological evolution.

Instead of improving only one candidate solution, an EA maintains a population of possible solutions and improves that population over many generations.



Core Idea



1. Each candidate solution represents a possible answer to the problem.

2. Better solutions are given a greater chance to contribute to the next generation, while variation creates new possibilities.

Real-World Example: Finding the Shortest Delivery Route (The Traveling Salesperson)

The Problem

- A delivery driver needs to visit **5 cities** and **return home.**
- Testing every possible order of cities takes **too long**, even for a computer.
- An evolutionary algorithm can **find a good (short) route quickly** by exploring many possible routes at the same time.



Why Use Evolutionary Algorithms?

- Enormous search space (many possible routes)
- Many local optima (short routes that aren't the best)
- No simple mathematical solution
- Complex objective function (e.g., distance, time, fuel, traffic, multiple constraints)

Evolutionary search **explores many different routes simultaneously and improves them over generations.**

How Evolutionary Algorithms Solve This Problem

1. Population

Multiple possible routes (solutions).

Each route is a candidate solution.

2. Evaluate

Calculate total distance for each route (fitness function).

420 km
380 km
500 km

Shorter distance = better fitness (higher chance to be selected).

3. Select

Choose better routes to be parents.

Better routes get a greater chance to reproduce.

4. Reproduce (Crossover)

Combine parts of two parent routes.

Create new routes by combining good parts.

5. Mutate

Randomly change parts of a route.

Make small random changes to explore new possibilities.

6. New Population

New and improved routes for the next generation.

Repeat evaluate → select → reproduce → mutate over many generations.

Result: A Short Delivery Route



After several generations, the algorithm finds a short (near-optimal) route.

Key Message: Evolutionary Algorithms search for good solutions by repeatedly applying **selection and variation to a population.**

Genetic Algorithms: Turning Solutions into Chromosomes

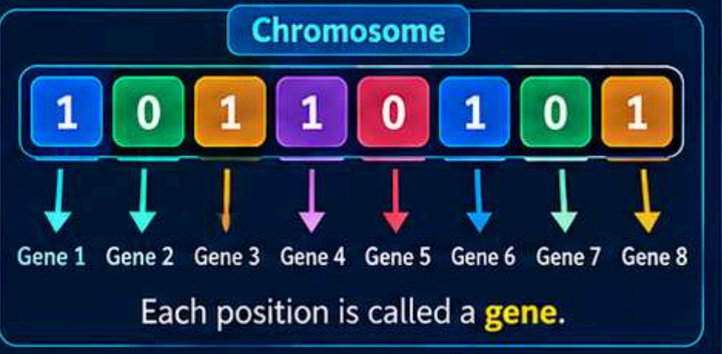
Genetic Algorithm (GA)

- A Genetic Algorithm is one of the best-known types of Evolutionary Algorithm.
- It represents each candidate solution as a **chromosome**.
- A population of chromosomes is evolved using **selection, crossover, and mutation** to find better solutions over many generations.

Chromosome Encoding

- A chromosome is an encoded representation of a solution.
- It is usually a string of symbols (often 0s and 1s), and each position is called a **gene**.
- Different encodings can be used depending on the problem (binary, integer, real-valued, or permutation).

Example (Binary Encoding)



How a Chromosome Represents a Solution (Example)

Problem: Select features for a machine learning model.

Gene Position	1	2	3	4	5	6	7	8	8
Feature	Age	Salary	Education	Experience	Location	Credit Score	Gender	Marital Status	
Chromosome	1	0	1	1	0	1	0	1	
Meaning	Use	Not Use	Use	Use	Not Use	Use	Not Use	Use	

This chromosome means: **use Age, Education, Experience, Credit Score, and Marital Status** as input features for the model.

Other Encoding Types (Depend ng on the Problem)

Binary Encoding

Uses 0s and 1s.

1 0 1 0 1

Used for:
- Feature selection
- On/off decisions

Integer Encoding

Uses integers.

3 1 4 2 5

Used for:
- Resource allocation
- Scheduling

Permutation Encoding

A permutation of items (e.g., city order in TSP).

3 1 5 2 4

Used for:
- Route optimization
- Task ordering

Real-Valued Encoding

Uses real numbers.

0.25 1.80 -0.50 2.10

Used for:
- Optimize continuous parameters (e.g., neural network weights)
- Engineering design

Key Message: A Genetic Algorithm encodes each candidate solution as a **chromosome** (a string of **genes**) and evolves a population of chromosomes to find **better solutions**.

Chromosome Encoding in Genetic Algorithms

Different Representations for Different Problems

1 What is a Chromosome?

- A chromosome is an encoded representation of a solution.
- It is usually a string of symbols, and each position is called a **gene**.
- A population of chromosomes evolves using **selection**, **crossover**, **and mutation** to find better solutions.

2 Types of Chromosome Encoding

Depending on the problem, chromosomes may contain:

Binary Values (0 or 1)

1 0 1 1 0 0

- Feature selection
- On/off decisions
- Rule generation

Integers

10 25 7 3

- Resource allocation
- Scheduling
- Discrete parameters

Real Numbers

0.1 0.5 2.3 -1.0

- Neural network parameters
- Continuous optimization

Symbols

A B C D

- Rule-based systems
- Grammar-based learning

Ordered Sequences

1 3 2 5 4

- Traveling Salesperson Problem (TSP)
- Scheduling
- Ordering / sequencing

3 Example: AI Parameter Optimization

Suppose an AI system must choose the best:
[Learning Rate, Hidden Units, Dropout]



Neural Network Model

Chromosome (Real-Valued)

Learning Rate	Hidden Units	Dropout
0.001	128	0.25

4 Multiple Individuals (Population)

Each chromosome represents one possible configuration of the AI model.

Individual	Learning Rate	Hidden Units	Dropout
Chromosome 1	0.001	128	0.25
Chromosome 2	0.01	256	0.40
Chromosome 3	0.005	64	0.30
Chromosome 4	0.002	512	0.20
...	...	...	...

5 How It Works in GA (for this Example)

Initial Population
Generate random model configurations (chromosomes).

Evaluate Fitness
Train the model and measure performance (e.g., validation accuracy).

Select Better Individuals
Choose the best-performing configurations as parents.

Crossover & Mutation
Create new configurations by combining and slightly changing parameters.

New Population
Repeat for many generations to find better model parameters.

Key Takeaways

- A chromosome encodes one candidate solution.
- Depending on the problem, it can contain binary values, integers, real numbers, symbols, or ordered sequences.
- In AI parameter optimization, a chromosome represents a set of model hyperparameters (e.g., [Learning Rate, Hidden Units, Dropout]).
- A population of chromosomes is evolved to find **better solutions** over many generations.

Important Design Question in Genetic Algorithms

How should the problem be encoded so that useful solutions can evolve?

The Design Question

How should the problem be **encoded** so that **useful solutions can evolve?**

- A chromosome is an encoded candidate solution.
- The encoding must allow **crossover and mutation** to create **meaningful** new solutions.
- Poor encoding can make the search difficult, while good encoding leads to **faster** and **better solutions**.

Different Chromosome Encodings

Depending on the problem, chromosomes may contain:

Binary Encoding (0 or 1)

1 0 1 1 0 0

- Feature selection
- On/off decisions
- Rule generation

Integer Encoding (Whole numbers)

10 25 7 3

- Resource allocation
- Scheduling
- Discrete parameters

Real-Valued Encoding (Decimal numbers)

0.1 0.5 2.3 -1.0

- Neural network parameters
- Engineering design
- Continuous optimization

Symbol Encoding (Categories)

A B C D

- Grammar-based systems
- Rule-based learning
- Symbolic regression

Ordered Sequence (Permutation)

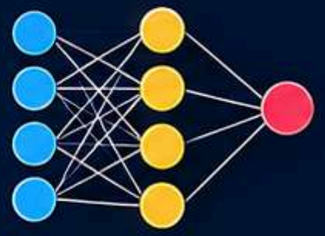
1 3 2 5 4

- Traveling Salesperson
- Scheduling
- Ordering / sequencing

Example: AI Parameter Optimization

Suppose an AI system must choose the best:
[Learning Rate, Hidden Units, Dropout]

Chromosome (Real-Valued)	Learning Rate	Hidden Units	Dropout
Individual 1	0.001	128	0.25
Individual 2	0.01	256	0.40
Individual 3	0.005	64	0.30
...	...	...	...



Each chromosome represents one possible configuration of the AI model.

Good vs Poor Encoding

Poor Encoding

- Solutions are hard to interpret.
- Crossover and mutation create invalid or meaningless solutions.
- Search becomes slow and inefficient.

0 0 1 1 1 0 ✗

May produce invalid or unusable solutions.

Good Encoding

- Solutions are meaningful and valid.
- Crossover and mutation can create useful new solutions.
- Leads to faster convergence and better results.

0 1 0 1 1 0 ✓

More likely to produce high-quality solutions.

Key Idea:

Chromosome = encoded candidate solution

A well-designed encoding allows the population to evolve **better solutions** over many generations.

Fitness Function: Measuring How Good a Solution Is

A fitness function gives every individual a numerical score representing its quality.

1 What is a Fitness Function?

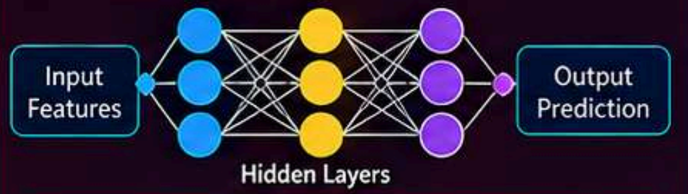
- The fitness function evaluates how good a solution (chromosome) is.
- It assigns a **numerical score** to each individual.
- **Higher fitness** means a better solution (the exact meaning of "better" depends on the problem).

2 What Does It Do in GA?

- Compares all solutions in the population.
- Helps select better individuals for reproduction.
- Guides the search towards high-quality solutions.

3 Example: AI Model Optimization

Suppose GA searches for the best neural network settings.



Chromosome (Model Configuration)

Learning Rate	Hidden Units	Dropout
0.001	128	0.25

4 Possible Fitness Criteria

A simple fitness measure could consider:

- **Validation Accuracy** — Higher accuracy is better.
- **Training Time** — Lower training time is better.
- **Model Size** — Smaller model size is better.
- **Computational Cost** — Lower computational cost is better.

5 Example Fitness Function

We can combine multiple factors into a single score. For example:

$$\text{Fitness} = \text{Accuracy} - \alpha \times (\text{Training Time}) - \beta \times (\text{Model Size}) - \gamma \times (\text{Computational Cost})$$

Where:

- Accuracy: validation accuracy (0 to 1)
- Training Time: time in seconds
- Model Size: number of parameters (in millions)
- Computational Cost: estimated cost
- α, β, γ: weights to balance the factors

6 Example Calculation

Assume $\alpha = 0.2$, $\beta = 0.1$, $\gamma = 0.05$

Chromosome	Accuracy (higher ↑)	Training Time (seconds)	Model Size (M)	Computational Cost	Fitness	Fitness (higher ↑)
A: [0.001, 128, 0.25]	0.92	300	1.2	0.5	0.92 – 0.2×300 – 0.1×1.2 – 0.05×0.5	-59.13
B: [0.01, 256, 0.40]	0.88	150	2.5	0.8	0.88 – 0.2×150 – 0.1×2.5 – 0.05×0.8	-29.10
C: [0.005, 64, 0.30]	0.90	80	1.0	0.3	0.90 – 0.2×80 – 0.1×1.0 – 0.05×0.3	**-15.12**

↳ Highest fitness (best solution)

7 Fitness Evaluation Process



8 Important Notes

- The fitness function should reflect the real goal of the problem.
- Different problems may maximize or minimize the fitness.
- You can combine multiple objectives using weighted sums or more advanced methods (e.g., multi-objective optimization).

9 Key Takeaways

- A fitness function measures how good a solution is.
- It gives a numerical score to each chromosome.
- For AI model optimization, it can include accuracy, training time, model size, and computational cost.
- A well-designed fitness function helps the GA find better solutions faster.

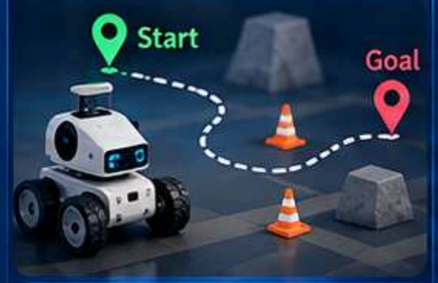
Key Message: The **fitness function** guides the evolutionary search by telling the algorithm which solutions are better, allowing it to select, reproduce, and evolve **high-quality solutions** over many generations.

AI Connection: Fitness Functions in Genetic Algorithms

Fitness functions can represent many different goals in real-world AI problems.

1 Robot Navigation

Find the shortest and safest path for a robot to reach a goal.



Fitness Function:
Minimize total navigation distance (and avoid obstacles).

2 Game Playing

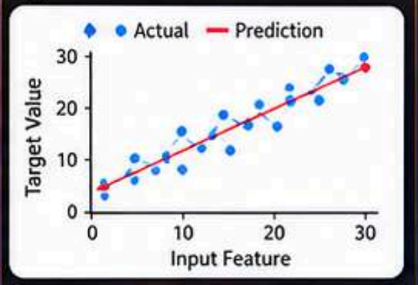
Find the best strategy to maximize the game score.



Fitness Function:
Maximize game-playing score (e.g., win more games).

3 Prediction Models

Find the best model parameters to reduce prediction error.



Fitness Function:
Minimize prediction error (e.g., Mean Squared Error).

4 Energy Consumption

Find system settings that minimize energy usage.



Fitness Function:
Minimize energy consumption (e.g., power in watts or total energy cost).

5 Scheduling

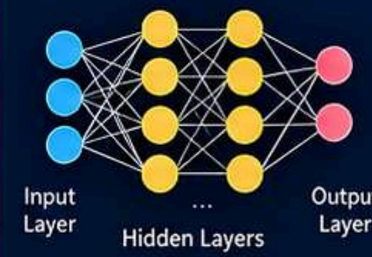
Find the best schedule with no conflicts and minimum delays.



Fitness Function:
Minimize scheduling conflicts and total delay (or maximize resource utilization).

6 Neural Network Architecture

Find the best network structure and hyperparameters.



Fitness Function:
Maximize validation accuracy (or balance accuracy with model size and training time).

General Example: AI Model Optimization

Chromosome = [Learning Rate, Hidden Units, Dropout]

Candidate	Learning Rate	Hidden Units	Dropout	Validation Accuracy (Fitness)
A	0.001	128	0.25	91%
B	0.01	256	0.40	94%
C	0.005	64	0.30	87%

If accuracy is the objective, **Candidate B** receives the strongest fitness **(94%)**.

Key Points about Fitness Functions

1. Fitness functions can represent many different goals in AI and optimization problems.
2. A **higher fitness** means a better solution.
3. Solutions with higher fitness have a greater chance of being selected for reproduction.
4. Common goals include minimizing distance, maximizing score, reducing error, minimizing energy, optimizing schedules, and designing efficient neural networks.
5. The GA does not need to know how to solve the problem — it only needs a way to **measure how good a candidate solution is**.

Key Message

The Genetic Algorithm does not need to know how to solve the problem — it needs a way to measure how good a candidate solution is.

- The fitness function provides this measure.
- It guides the selection process.
- It drives the evolution towards better solutions over many generations.

Selection in Genetic Algorithms: Who Gets to Reproduce?

After evaluating fitness, the algorithm must select individuals that will become parents.

1 What is Selection?

- Selection chooses individuals (chromosomes) from the current population to become parents for the next generation.
- It creates **evolutionary pressure** toward **better solutions** (higher fitness).
- Weaker individuals are not always removed because **maintaining diversity** can help discover better areas of the search space.

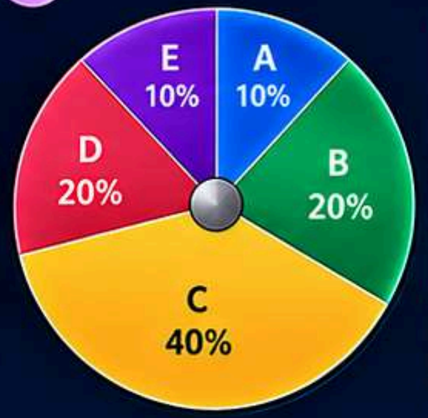
2 Why is Selection Important?

- Guides the search towards high-quality solutions.
- Allows good solutions to pass their genetic information.
- Maintains diversity (not only the best are selected).
- Helps explore new and better areas of the search space.

3 Roulette-Wheel Selection

- Think of a wheel divided according to fitness.
- A higher-fitness individual receives a **larger section** of the wheel.
- Individuals with higher fitness have a greater chance of being selected, but even lower-fitness individuals still have a chance.

4 Roulette-Wheel Illustration



Fitness Proportions

- A : 10%
- B : 20%
- C : 40%
- D : 20%
- E : 10%

Individual **C** has the largest section, so it has the highest chance of being selected.

5 Example: Roulette-Wheel Selection

Individual	Fitness	Selection Probability (Fitness / Total Fitness)
A	10	0.10 (10%)
B	20	0.20 (20%)
C	40	0.40 (40%)
D	20	0.20 (20%)
E	10	0.10 (10%)
Total	100	1.00 (100%)



Spin the wheel and select one individual based on the **probability proportional to its fitness.**

6 Selection Outcome (Random Spin Examples)

Random Number (0 to 1)	Selected Individual	Corresponding Range
0.03	A	0.00 – 0.10
0.25	B	0.10 – 0.30
0.67	C	0.30 – 0.70
0.85	D	0.70 – 0.90
0.96	E	0.90 – 1.00

7 Selection in the GA Process

1. Evaluate fitness of all individuals
2. Select parents (using roulette-wheel selection)
3. Apply crossover and mutation to create offspring
4. Form the next generation population

Key Takeaways

- Selection creates evolutionary pressure toward better solutions.
- In roulette-wheel selection, higher-fitness individuals have a larger chance of being selected.
- Lower-fitness individuals can still be selected, which helps maintain diversity.
- Maintaining diversity prevents premature convergence and allows exploration of better solutions.

Roulette-Wheel Selection in Genetic Algorithms

Better fitness → Higher probability of selection, but even lower-fitness individuals can still be selected.

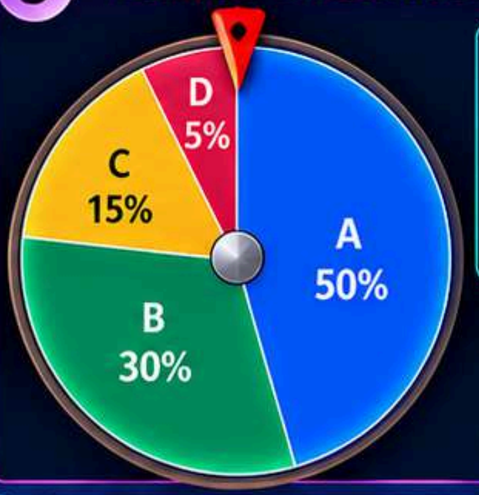
1 What is Roulette-Wheel Selection?

- A selection method where individuals are chosen with **probability proportional** to their **fitness**.
- Think of a wheel divided according to fitness.
- A higher-fitness individual receives a **larger section** of the wheel.
- The wheel is spun **randomly** to pick parents for the next generation.

2 Example: Fitness and Selection Chance

Individual	Fitness	Selection Chance (Approx.)
A	50	High (≈ 50%)
B	30	Medium (≈ 30%)
C	15	Lower (≈ 15%)
D	5	Very Low (≈ 5%)

3 Roulette-Wheel Illustration



Fitness Proportions

- A : 50%
- B : 30%
- C : 15%
- D : 5%

The wheel is divided according to fitness. A **larger section** means a **higher chance** of being selected.

4 Random Selection Process

Step 1: Spin the wheel (random number between 0 and 1).

Step 2: Find the section that the random number falls into.

Step 3: Select the corresponding individual as parent.

5 Example Spin Results

Random Number (0 to 1)	Selected Individual	Explanation
0.12	A	Falls in A's section (0.00 – 0.50)
0.41	A	Falls in A's section (0.00 – 0.50)
0.63	B	Falls in B's section (0.50 – 0.80)
0.82	C	Falls in C's section (0.80 – 0.95)
0.97	D	Falls in D's section (0.95 – 1.00)

6 Why Allow Lower-Fitness Selection?

- Maintains population diversity.
- Prevents premature convergencê (getting stuck in a local optimum).
- Allows exploration of new and potentially better areas of the search space.
- Even lower-fitness individuals can still **occasionally** be selected.

Key Points

- Better fitness → Higher probability of selection.
- Lower-fitness individuals can still be selected ((due to randomness).
- This randomness helps preserve diversity and encourages **exploration**.

Key Message: Roulette-wheel selection creates **evolutionary pressure** toward **better solutions** while **maintaining diversity** in the population.

Tournament Selection + Simple GA Workflow

Tournament selection chooses the best individual from a random group. The selected parents are then used in the GA workflow to create new generations.

1 What is Tournament Selection?

- Tournament selection **randomly chooses** a small group of individuals from the population.
- The individual with the **highest fitness** in the group becomes a parent.
- It is simple, effective, and keeps diversity (not always the global best).
- It creates **evolutionary pressure** toward better solutions while still giving lower-fitness individuals a chance.

2 Example: Tournament Selection

Step 1: Population

A B C D E F

(6 individuals)

Step 2: Randomly choose a tournament group (e.g., size = 3)

B D F

(Randomly selected)

Step 3: Compare fitness values

Individual	B	D	F
Fitness	72	91	65

Step 4: Winner

D — Individual D has the highest fitness (91). D becomes a parent.

3 How Tournament Selection Works

Randomly select k individuals (e.g., k = 3) from the population.

↓

Compare their fitness values.

↓

Select the individual with the highest fitness as a parent.

↓

Repeat to select more parents (for mating).

4 Another Example

Population and fitness values:

Individual	Fitness
A	50
B	72
C	60
D	91
E	40
F	65

Random tournament (k = 3): B, D, F

Fitness: B = 72, D = 91, F = 65

Winner: D → (highest fitness)

5 Simple Genetic Algorithm (GA) Workflow

1. **Initialize Population** — Create an initial set of random chromosomes (solutions).
2. **Evaluate Fitness** — Calculate fitness for each individual.
3. **Select Parents** — Use tournament selection (or other methods) to choose parents.
4. **Crossover (Recombination)** — Combine two parents to create new offspring.
5. **Mutation** — Randomly change some genes to maintain diversity.
6. **Create New Population** — Replace old population with new offspring (keep best if needed).
7. **Repeat Until Stop** — Repeat for a fixed number of generations or until a good solution is found.

6 Key Takeaways

- Tournament selection is simple and effective.
- It selects the best individual from a random group.
- It maintains diversity compared to always picking the global best.
- It can be easily combined with crossover and mutation.
- It is widely used in Genetic Algorithms for real-world AI optimization problems.

Key Message: Tournament selection creates **evolutionary pressure** toward better solutions while **maintaining diversity**, and it is an important step in the overall Genetic Algorithm workflow.

Tournament Selection in Genetic Algorithms

Select the best individual from a random group — Simple, effective, and widely used.

1 What is Tournament Selection?

- Tournament selection **randomly chooses** a small group of individuals from the population.
- The individual with the **highest fitness** in the group becomes a parent.
- It is simple, effective, and maintains diversity (not always the global best).

2 Example: One Tournament

Population (6 individuals):

A B C D E F

(A, B, C, D, E, F)

Randomly select 3 individuals (tournament size k = 3):

B D F

(Random tournament)

Individual	B	D	F
Fitness	72	91	65

Winner: D

(highest fitness → becomes a parent)

3 Steps in Tournament Selection

1. Randomly select k individuals from the population.
2. Compare their fitness values.
3. Select the individual with the highest fitness as a parent.
4. Repeat the process to select more parents (for mating).

4 Different Tournament Outcomes

Tournament (k = 3)	Selected Individual	Fitness Values (in Tournament)	Winner
A, B, C	B	A = 50 B = 72 C = 60	B (72)
C, D, E	D	C = 60 D = 91 E = 40	D (91)
A, E, F	A	A = 50 E = 40 F = 65	F (65)

5 Why Tournament Selection Is Popular

- **Simple to implement** — Easy to understand and code.
- **Does not require fitness probabilities** — Avoids calculating selection probabilities.
- **Works well with large populations** — Scales efficiently.
- **Selection pressure can be controlled** using tournament size (k).

6 Effect of Tournament Size (k)

k = 2 (Small Tournament)

- Lower selection pressure
- More diversity
- Higher chance of selecting lower-fitness individuals

More exploration of the search space.

k = 5 (Large Tournament)

- Higher selection pressure
- Stronger preference for high-fitness solutions
- Lower diversity

More exploitation (focus on better solutions).

7 Key Takeaways

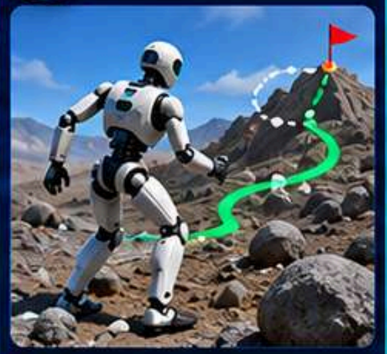
- Tournament selection is simple and effective.
- It does not require fitness probabilities.
- It works well with large populations.
- Selection pressure can be controlled using tournament size:
  - **Larger tournament (k)** → stronger preference for high-fitness solutions.
  - **Smaller tournament (k)** → more diversity.

Key Message: Tournament selection creates **evolutionary pressure** toward better solutions while allowing some **diversity**, and the **tournament size (k)** controls the balance between **exploration** and **exploitation**.

Why This Matters in Modern AI

Evolutionary techniques are powerful when the AI system must search through a huge number of possibilities and there is no simple formula to find the best solution.

1 Robot Control



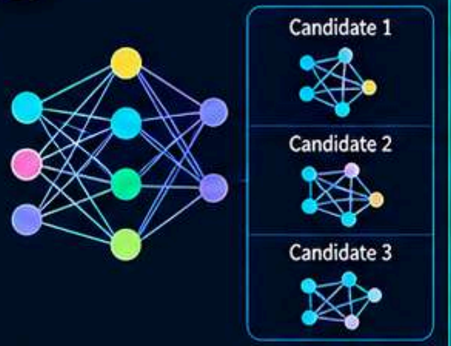
- Find optimal walking strategies.
- Adapt to changing environments.
- Evolve control policies for better stability and efficiency.

2 Game Strategies



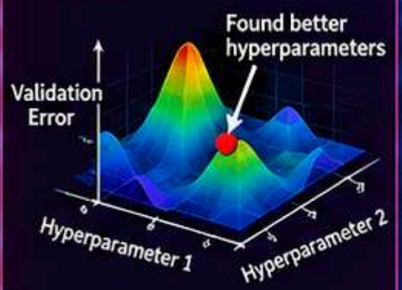
- Discover strong strategies for complex games.
- Learn effective policies through self-play.
- Used in game AI and simulations.

3 Neural Architecture Search



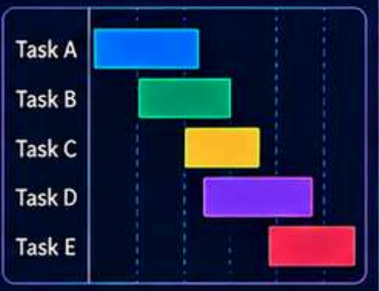
- Automatically design neural network architectures.
- Search for models with higher accuracy and lower complexity.
- Reduces manual trial-and-error.

4 Hyperparameter Optimization



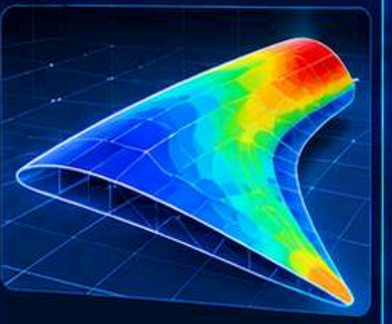
- Search for the best settings (e.g., learning rate, batch size).
- Can significantly improve model performance.
- Works well for complex, multi-parameter spaces.

5 Scheduling



- Optimize task schedules (e.g., in computing, logistics).
- Minimize completion time, cost, or resource usage.
- Handle complex constraints and large search spaces.

6 Engineering Design



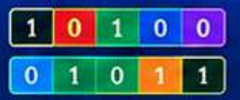
- Optimize shapes and structures (e.g., aircraft wings).
- Improve performance (e.g., strength, efficiency).
- Explore innovative designs beyond human intuition.

7 General Idea

Instead of explicitly programming the final solution, we **define the key components** and allow the **evolutionary search process** to discover increasingly effective solutions on its own.

Define and Let Evolution Discover the Solution

Representation



How to encode a candidate solution (e.g., binary string, real numbers, etc.).

\+

Fitness Function



How to measure how good a solution is (e.g., accuracy, cost, distance, efficiency).

\+

Evolutionary Operators



Selection, crossover, and mutation to create new solutions.

Evolving Solutions



The search process discovers increasingly better solutions over generations.

8 Key Takeaways

- Evolutionary methods are useful when the search space is very large.
- They can solve complex optimization problems where traditional methods struggle.
- They are widely used in modern AI applications.
- We define the representation, fitness, and evolutionary operators, and let the search process discover good solutions.

Key Message: Evolutionary techniques provide a **flexible** and **powerful** way to search through huge solution spaces and are widely applicable in modern AI for **real-world optimization problems**.

Remember: Key Questions in Genetic Algorithms

Each component of a Genetic Algorithm answers a specific question. Together, they allow a population to explore, compete, adapt, and improve across generations.

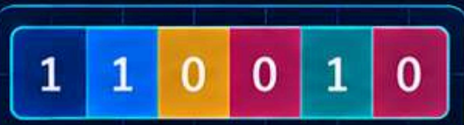
1 Encoding

What does a solution look like?

Define how to represent a candidate solution (chromosome).

Example encodings:

- Binary string (e.g., 110010)
- Integer values (e.g., [3, 1, 4])
- Real numbers (e.g., [0.5, 2.7])
- Symbols or permutations (e.g., [A, C, B, D])



Chromosome
(representation of a solution)

2 Fitness

How good is this solution?

Calculate a numerical score that measures how well the solution solves the problem.

Examples:

- Higher game score
- Lower prediction error
- Shorter path length
- Lower cost / energy
- Better schedule quality
- Higher model accuracy



Fitness Score

3 Selection

Which solutions should reproduce?

Choose better individuals as parents based on fitness (e.g., roulette-wheel selection, tournament selection).

Common methods:

- Roulette-wheel selection
- Tournament selection
- Rank selection
- Stochastic uniform selection



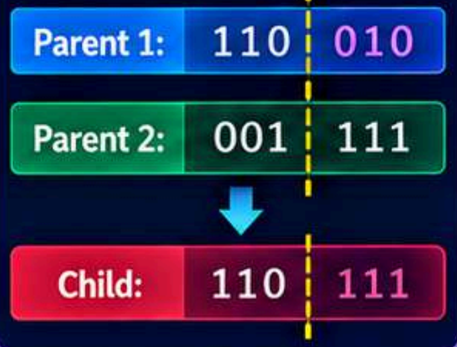
Select high-fitness individuals as parents

4 Crossover

How can successful solutions be combined?

Combine information from two parent solutions to create new offspring (exchange genes).

Example (single-point crossover):



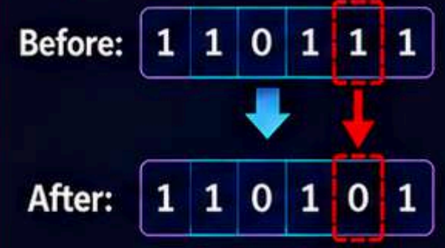
Crossover combines good traits from both parents.

5 Mutation

How do we introduce new possibilities?

Randomly change a small part of the chromosome to maintain diversity and explore new areas of the search space.

Example (bit mutation):



- Introduces new genetic variation.
- Helps explore new regions.
- Prevents premature convergence.

Key Message: Encoding, fitness, selection, crossover, and mutation work together to allow a population to **explore, compete, adapt, and improve** across generations, finding increasingly effective solutions.

Question 1: Laboratory Sample Processing

A diagnostic laboratory uses a Genetic Algorithm to arrange four processing stages, T1, T2, T3, and T4. For this exercise, the stages may be performed in any order, but every stage must appear exactly once in each chromosome. The objective is to minimize the total equipment-changeover cost. The population is limited to **4 chromosomes**.

The costs between consecutive locations or tasks are given below:

From / To	T1	T2	T3	T4
T1	-	6	3	8
T2	6	-	5	2
T3	3	5	-	4
T4	8	2	4	-

Initial population

C1 = T1 T2 T3 T4

C2 = T3 T1 T4 T2

C3 = T2 T4 T1 T3

C4 = T4 T3 T2 T1

a) Define a fitness function that assigns a higher fitness to a sequence with a lower total cost. Calculate the sequence cost and fitness of every chromosome in the initial population.

b) Choose and apply suitable parent-selection, crossover, and mutation methods to construct the next generation. Show the selected parents, crossover positions, offspring before mutation, and offspring after mutation.

c) Check every offspring for repeated or missing stages. Discard any invalid offspring, or explain why none are invalid. Present exactly four valid chromosomes in the next generation and name all genetic operators used. If necessary, retain valid parents or generate additional valid offspring.

Assumptions: Count only transitions between consecutive genes; do not add a return journey or an initial depot cost. State any random choices, tie-breaking rules, and operator settings used. Different valid next generations are acceptable if the procedure is shown consistently.

Question 2: Greenhouse Inspection Route

An agricultural robot must inspect four greenhouse sections, R1, R2, R3, and R4, once each. A Genetic Algorithm is used to find an inspection order with low travel cost. Any section may be visited first, and every chromosome must contain all four section labels without repetition. At most **4 chromosomes** may be retained in the population.

The costs between consecutive locations or tasks are given below:

From / To	R1	R2	R3	R4
R1	-	5	9	2
R2	5	-	3	7
R3	9	3	-	4
R4	2	7	4	-

Initial population

C1 = R1 R3 R2 R4

C2 = R2 R1 R4 R3

C3 = R3 R4 R1 R2

C4 = R4 R2 R3 R1

a) Propose an appropriate fitness function for this minimization problem. Compute the total travel cost and corresponding fitness for each starting chromosome, then rank the population by fitness.

b) Produce offspring using a suitable selection strategy and a permutation-compatible crossover method. Apply a swap mutation to at least one offspring and show the chromosome before and after the swap.

c) Identify and remove any invalid offspring. Build a next generation of exactly four valid chromosomes while preserving one best chromosome from the initial population unchanged. State the survivor-selection rule and explain the purpose of preserving that chromosome.

Assumptions: Count only transitions between consecutive genes; do not add a return journey or an initial depot cost. State any random choices, tie-breaking rules, and operator settings used. Different valid next generations are acceptable if the procedure is shown consistently.

Question 3: Printing Machine Job Order

A printing workshop has five independent jobs, J1, J2, J3, J4, and J5. Changing the machine from one job to another incurs the setup cost shown below. A Genetic Algorithm will optimize the job order. Each job must occur exactly once in a chromosome, and the maximum retained population size is **4 chromosomes**.

The costs between consecutive locations or tasks are given below:

From / To	J1	J2	J3	J4	J5
J1	-	4	8	3	6
J2	4	-	5	7	2
J3	8	5	-	4	3
J4	3	7	4	-	5
J5	6	2	3	5	-

Initial population

C1 = J1 J3 J2 J5 J4

C2 = J2 J4 J1 J3 J5

C3 = J3 J5 J4 J2 J1

C4 = J4 J1 J5 J3 J2

a) Develop a fitness function suitable for minimizing setup cost. Calculate the cost and fitness of each chromosome, showing the consecutive transitions used in your calculations.

b) Apply a parent-selection method and an order-preserving crossover to generate new job sequences. Mutate at least one offspring using either inversion or swap mutation. Clearly identify the selected parents, cut positions, and mutation positions.

c) Validate all generated sequences and discard any that omit or repeat a job. Use a stated replacement policy to form a population of exactly four valid chromosomes. Report the final chromosomes and their costs, and explain whether your chosen mutation operator preserves chromosome validity.

Assumptions: Count only transitions between consecutive genes; do not add a return journey or an initial depot cost. State any random choices, tie-breaking rules, and operator settings used. Different valid next generations are acceptable if the procedure is shown consistently.

Question 4: Campus Maintenance Schedule

A maintenance team must visit five service points, S1, S2, S3, S4, and S5, during one shift. The visits have no precedence restrictions. The team uses a Genetic Algorithm to reduce the total movement cost between successive service points. A valid chromosome includes every point exactly once, and the population may contain no more than **4 chromosomes**.

The costs between consecutive locations or tasks are given below:

From / To	S1	S2	S3	S4	S5
S1	-	7	4	6	3
S2	7	-	2	5	8
S3	4	2	-	3	6
S4	6	5	3	-	4
S5	3	8	6	4	-

Initial population

C1 = S1 S3 S5 S2 S4

C2 = S2 S4 S1 S5 S3

C3 = S3 S2 S5 S4 S1

C4 = S5 S1 S4 S3 S2

a) Choose a fitness function that favors lower-cost schedules and evaluate all four chromosomes. Identify the best chromosome under your chosen function.

b) Select two parents using a stated selection method. First demonstrate ordinary one-point crossover on these permutation chromosomes and check the resulting offspring. If any are invalid, discard them and use a permutation-compatible crossover to generate valid replacements. Apply a suitable mutation to at least one valid offspring.

c) List all rejected offspring and explain the reason for each rejection; if none are rejected, state why. Form a final population of exactly four valid chromosomes using a clearly stated survivor-selection policy. Explain why a repeated location within a chromosome differs from two identical valid chromosomes in a population.

Assumptions: Count only transitions between consecutive genes; do not add a return journey or an initial depot cost. State any random choices, tie-breaking rules, and operator settings used. Different valid next generations are acceptable if the procedure is shown consistently.

Question 1: Connecting a Network

A computer player competes in a two-player board game in which each player tries to connect opposite edges of a large hexagonal board. On each turn, a player places one piece on an empty cell. The entire board is visible, and the rules determine the effect of every legal move. Hundreds of placements may be available, making exhaustive analysis impractical. The program has no reliable formula for judging unfinished positions, but it can play many simulated continuations using a simple move policy and record who wins. It has two seconds to choose each move.

a) Recommend a suitable simulation-based search algorithm. Explain why it fits the large number of legal moves and the lack of a strong evaluation function for unfinished positions.

b) Define a state representation, the available actions, the transition model, the terminal-state conditions, and the agent's objective for this game.

c) Explain how repeated simulations would help the agent compare candidate placements within the two-second limit. Describe how results are stored and updated, how new moves receive attention, and how the final move is selected.

Question 2: Capturing Rival Flags

In a digital arena, two players alternately move units to capture the opposing flag. Every unit position, obstacle, and legal action is visible. A move may reposition a unit, block a passage, or capture an opposing unit, so the number of future action sequences grows rapidly. The game engine can simulate legal moves and complete trial matches, but the human player's next choice is not known. The AI can afford at most 5,000 simulated matches before acting. A few moves look promising after early trials, while several alternatives have barely been tested.

a) Identify an appropriate search method for allocating this simulation budget. Justify its suitability without assuming that the opponent's future actions are known.

b) Formulate the decision problem by specifying the state, legal actions, state-transition rules, and winning objective. Explain why the state must identify whose turn it is.

c) Describe how the method should balance further testing of promising moves against investigation of less-tested alternatives. Explain how simulated wins, losses, and draws can influence the decision, and state one limitation of relying on a simple opponent policy during simulations.

Question 3: Competing for Trade Routes

An AI participates in a two-player trading game with a fully visible map. On a turn, a player may claim a route, collect resources, or upgrade a trading post. Actions change future opportunities for both players, and a match ends after a fixed number of rounds. The player with the higher score wins. The search tree is too large to examine completely, but a simulator can finish trial games using inexpensive move rules. The AI receives a variable amount of thinking time: sometimes half a second and sometimes five seconds. It must return a legal move whenever time expires.

a) Choose a search algorithm suited to repeated trial games and a variable time allowance. Explain why being able to improve a decision as more computation becomes available is valuable here.

b) Specify the information needed in a game state, the action set, the transition model, and an objective based on the final result. Include any information needed to determine when the game ends.

c) Outline how the algorithm builds and updates its search tree from trial results. Explain how it can return a move when interrupted, how extra thinking time may help, and why more simulations do not guarantee the best possible move.

Question 4: Controlling an Orbital Arena

Two players command fleets in a turn-based space arena. Each turn permits one fleet movement or defensive action, and all positions and resources are visible. There are many legal actions and long sequences of possible replies. The simulator applies the game rules exactly and can generate trial battles until one side wins or the turn limit produces a draw. To meet a strict response deadline, the AI samples only a small fraction of possible continuations. One aggressive opening wins several early trials, but those trials contain mostly weak defensive replies.

a) Recommend a suitable search algorithm and justify using sampled continuations rather than attempting to construct the complete game tree.

b) Identify the states, actions, transition model, terminal conditions, and agent objective. Distinguish uncertainty about the opponent's chosen action from randomness in the game's transition rules.

c) Explain how trial outcomes should update the estimates associated with earlier moves in the search tree. Discuss why the early success of the aggressive opening could be misleading and how further exploration and a stronger simulation policy could improve the decision within the time limit.